Queen Mary
University of London

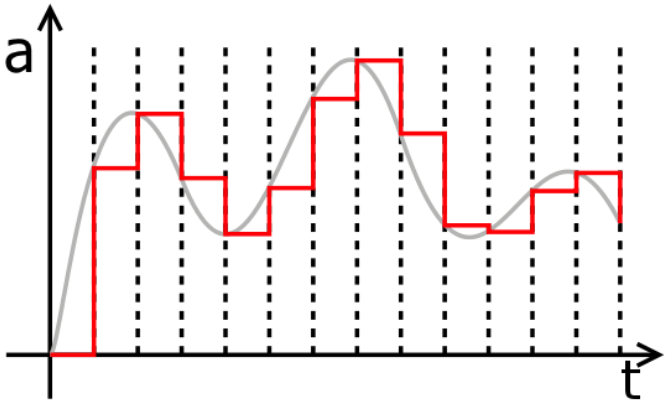
EBU5376
Digital Signal Processing

Sampling and Quantization

Module Organizer: Dr Yixuan Zou yixuan.zou@qmul.ac.uk

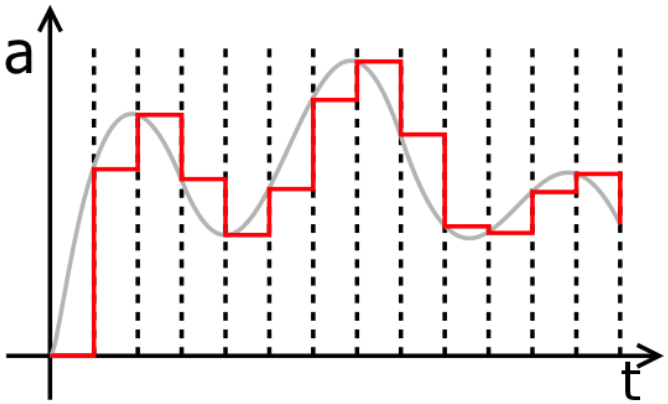
Outline

- Analog to Digital Converter (ADC)
  - Sampling
  - Quantization
- Digital to Analog Converter (DAC)
  - Zero-order Hold



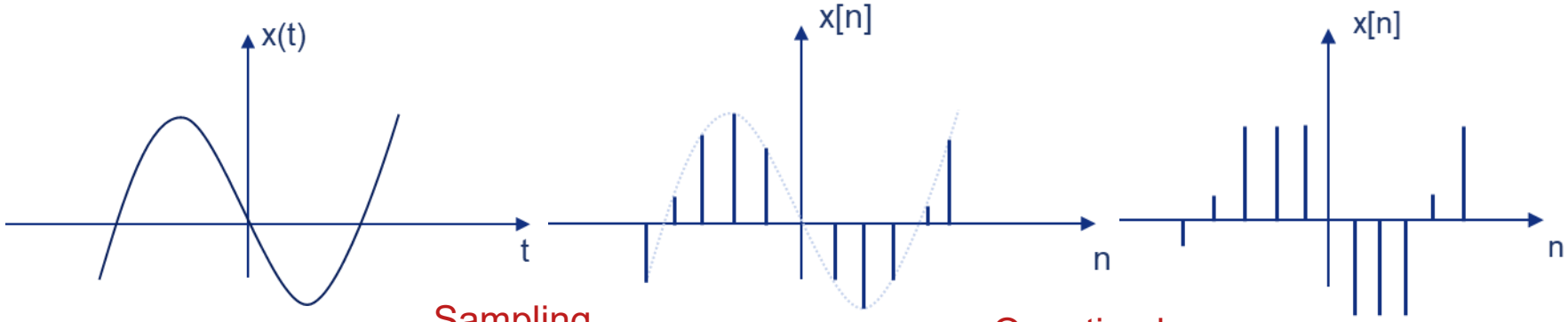
Outline

- Analog to Digital Converter (ADC)
  - Sampling
  - Quantization
- Digital to Analog Converter (DAC)
  - Zero-order Hold



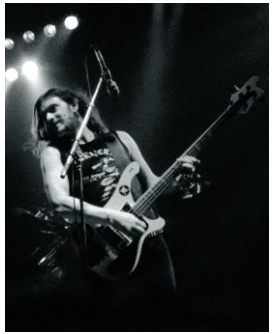
Classification of Signals (recap)

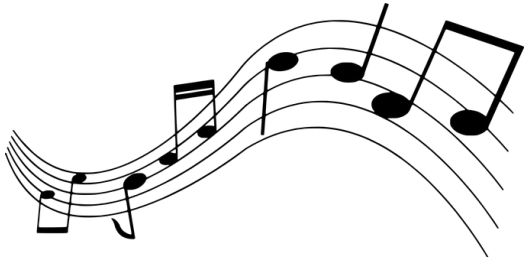
- From Analog to Digital
  - Analog to Digital Converter (ADC)



Analog Signal $x(t)$ → Sampling → Sampled Signal $x(nT_s)$ → Quantized → Digital Signal $x[n]$

Example application – audio processing





Microphone receives sound → Analog to Digital Converter → Digitally edit soundwave → Digital to Analog Converter → Speaker outputs sound

Usually also analog filtering here

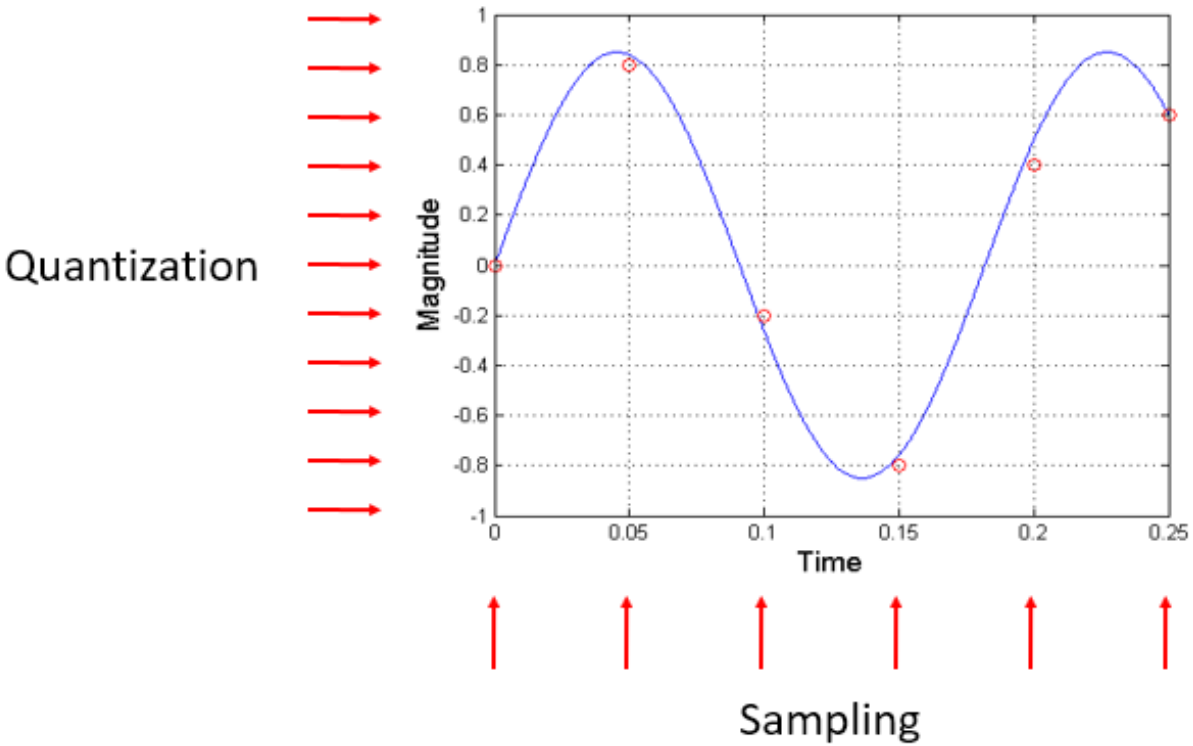
Why are we sampling/quantising?

- Sampling
  - A continuous signal has a value at any point in time.
  - A digital computer can only store a finite number of values.
  - Turn our continuous signal into a series of (usually) evenly spaced samples.
- Quantizing:
  - *A theoretical continuous signal has infinite precision.*
  - A digital computer can only store a finite number of decimal points.
  - In reality our measuring equipment has a finite precision.
  - Below that precision the measurement is really “noise”.

Sample, Quantise, Digitise

- Real world (analog) signal (can't be stored/processed on computer):
  - Value for any time t.
  - Does not have a specific "precision".
- Sampling:
  - Take a number of "samples" at time t_1, t_2, t_3,…
  - Usually regularly spaced (time series)
- Quantisation:
  - Round signal to some "precision"
- Sampling + Quantisation = Digitisation (can be stored)

Quantization vs. Sampling



Example 1 - Sample

- Consider the basic sine function with frequency 1Hz $x(t) = \sin(2\pi t)$.
- What do you get over one second if you sample with interval $T_s = 0.25\ s$?

Example 1 - Sample

- Consider the basic sine function with frequency 1Hz $x(t) = \sin(2\pi t)$.
- What do you get over one second if you sample with interval $T_s = 0.25\ s$?

	$x(nT_s)$	
• n=0, t=0	$x(0) = \sin(0)$	= 0
• n=1, t=0.25	$x(0.25) = \sin(0.5\,\pi)$	= 1
• n=2, t=0.5	$x(0.5) = \sin(\pi)$	= 0
• n=3, t=0.75	$x(0.75) = \sin(1.5\,\pi)$	= -1
• n=4, t=1	$x(1) = \sin(2\,\pi)$	= 0

Example 1 - Quantise

- Consider the basic sine function with frequency 1Hz $x(t) = \sin(2\pi t)$.
- What do you get over one second if you quantise $x(nT_s)$ to the nearest value from -1.2, -0.6, 0, 0.6,1.2?
- x(0) =0 x[0]= sin(0)= 0
- x(0.25)= 1 x[1]= sin(0.5 π) = 1.2
- x(0.5)= 0 x[2]= sin(π)= 0
- x(0.75)= -1 x[3]= sin(1.5 π) = -1.2
- x(1) =0 x[4]= sin (2 π) = 0

Example 2

- Consider the continuous signal $x(t) = \sin\left(\frac{t}{2\pi}\right)$.

  What do you get if you **sample** with interval $T_s = 0.25\ s$ for one second and **quantise** to the nearest value from 0, 0.05, 0.1, 0.15, 0.2?

- n=0, t=0 x(0)= sin(0)= 0
- n=1, t=0.25 x(0.25)= sin(0.175/π) = 0.05567…
- n=2, t=0.5 x(0.5)= sin(0.25/π)= 0.07949…
- n=3, t=0.75 x(0.75)= sin(0.375/π) = 0.11908…
- n=4, t=1 x(1)= sin (0.5/π) = 0.15848…

Sampling outputs

Example 2

- Consider the continuous signal $x(t) = \sin\left(\frac{t}{2\pi}\right)$.

  What do you get if you **sample** with interval $T_s = 0.25\ s$ for one second and **quantise** to the nearest value from 0, 0.05, 0.1, 0.15, 0.2?

x(0)=0	x[0]=0
x(0.25)= 0.05567…	x[1]=0.05
x(0.5)= 0.07949…	x[2]=0.1
x(0.75)= 0.11908…	x[3]=0.1
x(1)= 0.15848…	x[4]=0.15
Sampling outputs	Quantization outputs

More on Sampling and Quantization

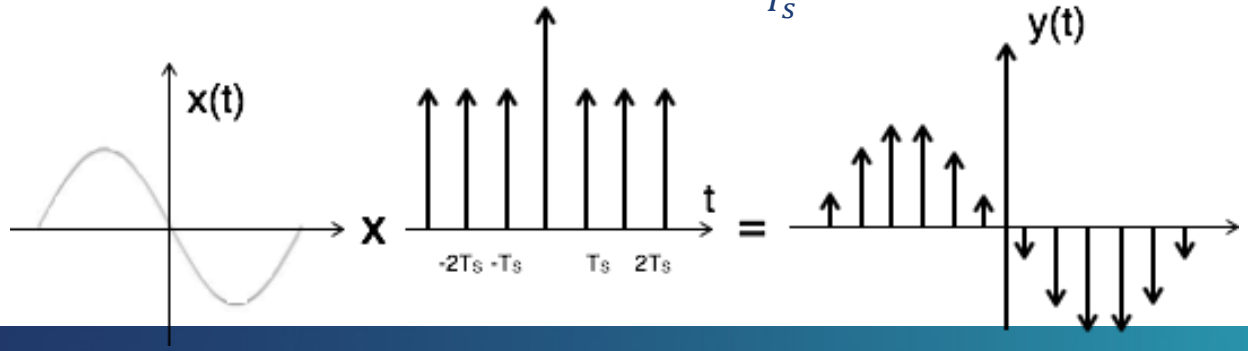
- <u>Sampling</u>
  - Impulse Sampling
  - Natural Sampling
  - Flat-top Sampling
- <u>Quantization</u>
  - Bit-based quantization
  - Quantization error

Impulse Sampling

- So far we are talking about the ideal sampling method, known as **impulse sampling (ideal sampling)**
- This is achieved by multiplying the input signal with <u>a train of impulses</u>
- Mathematical Representation of the ideal sampling process:

$$y(t) = x(t) \cdot \sum_{n=-\infty}^{\infty} \delta(t - nT_s)$$

  - Here T_s is the sampling interval and $f_s = \frac{1}{T_s}$ is the sampling frequency

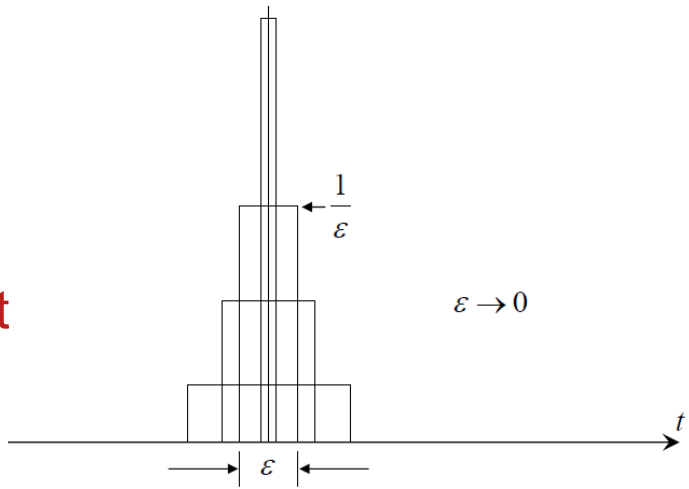


Impulse Sampling

- **Impulse sampling** is ideal but impractical

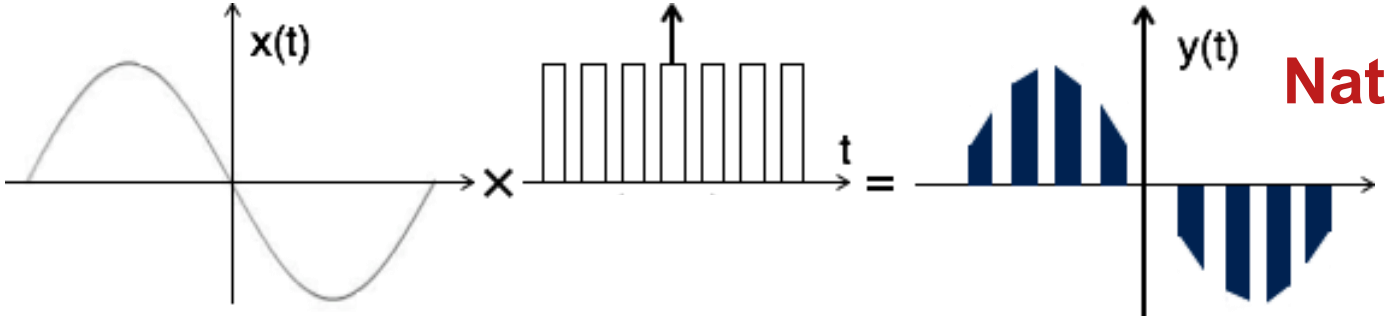
$$\delta(t) \equiv \begin{cases} 0 & t \neq 0 \\ \int_{-\varepsilon}^{\varepsilon} dt\, \delta(t) \equiv 1 & t = 0 \end{cases}$$

Dirac delta function is difficult to realize in practical systems

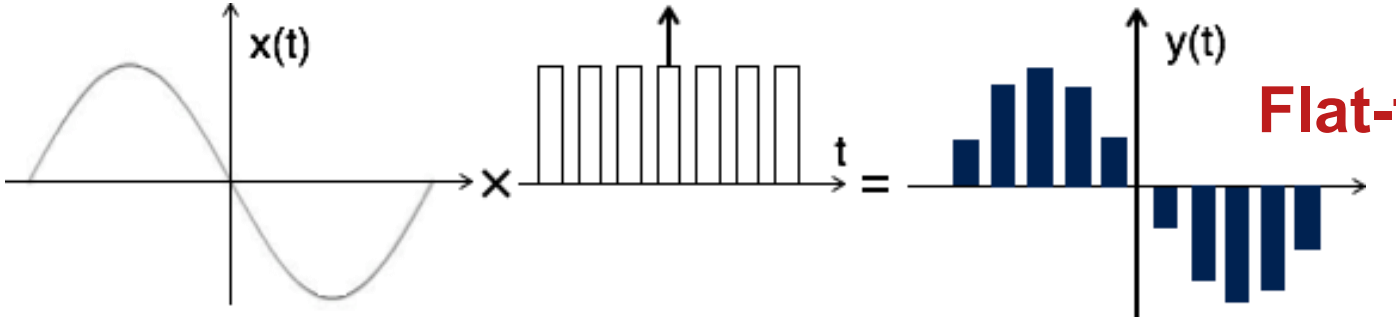


- To obtain a precise measurement at a specific time instant, we want ε to be as small as possible, i.e., $\varepsilon \to 0$
- This means, the amplitude of the impulse will become very large, i.e. $\delta(0) \to \infty$
- Also, the sampled signals have very short duration and are hard to capture.

Natural Sampling & Flat-top Sampling

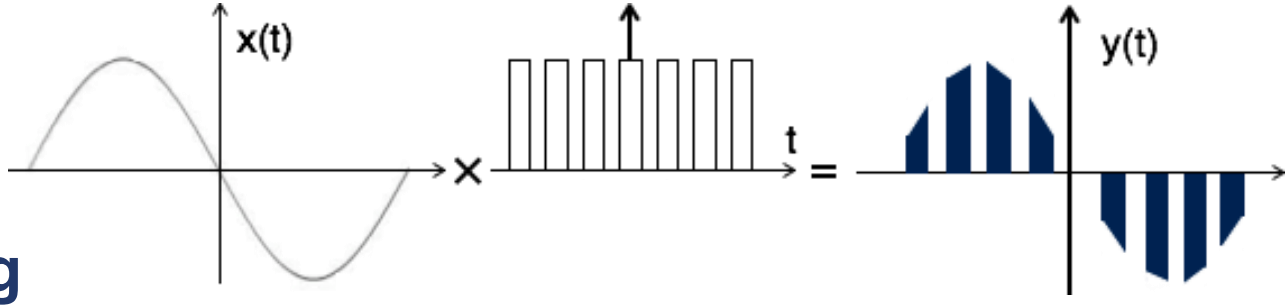


Natural Sampling



Flat-top Sampling

Sampling



➢ Natural Sampling

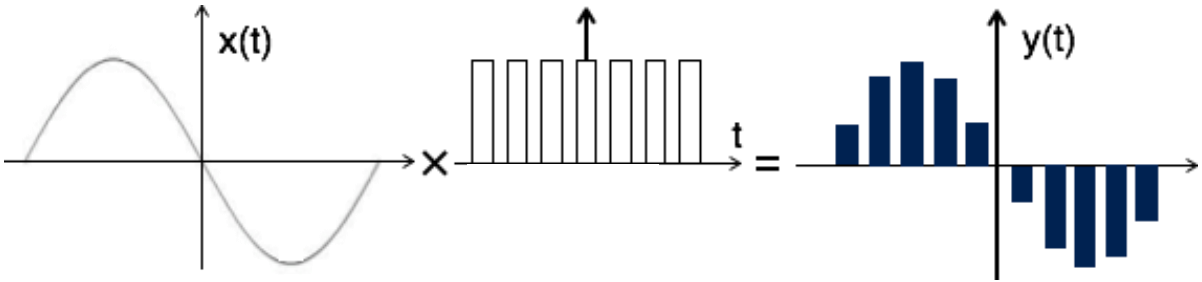
- It is a practical method that approximates ideal sampling.
- The analog input $x(t)$ is multiplied by **a train of uniformly spaced, rectangular pulses**.
- The width of each sampling pulse τ must satisfy $0 < \tau < T_s$, where T_s is the sampling period
  - If $\tau \to 0$, natural sampling approaches **impulse sampling**
  - If $\tau = T_s$, natural sampling becomes continuous multiplication (not periodic sampling)

Sampling

➢ Problems with Natural Sampling

- ❖ **Difficulty in Digitization**: Natural sampling produces continuously varying signals, making it harder for ADCs to process and digitize.
- ❖ **Complex Circuit Design**: Requires more advanced circuits to track continuous variations, increasing system complexity.
- ❖ **Noise Sensitivity**: Continuous signal variations can make the sampled signal more susceptible to noise.

Sampling

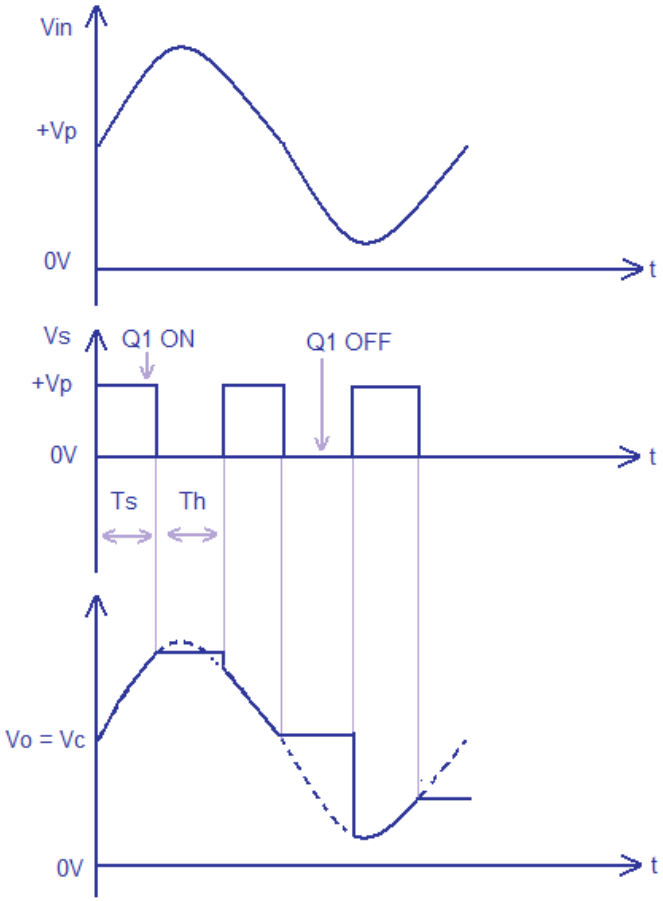


➢ Flat-top Sampling

- ❖ Flat-top sampling is a technique where the amplitude of each sample is held constant over a small duration after the sampling instant.
- ❖ The sampled signal appears as a staircase-like waveform, where each flat region corresponds to the instantaneous value of the signal at the sampling instant ($t = nT_s$).
- ❖ This is achieved using a sample-and-hold circuit.

Flat-top Sampling

- Sample and Hold
  - A type of analog circuit
  - Typically used in ADCs for achieving flat-top sampling
  - It **samples** (captures, takes) the voltage of a continuously varying analog signal and **holds** (locks, freezes) its value at a constant level for a specified minimum period of time.

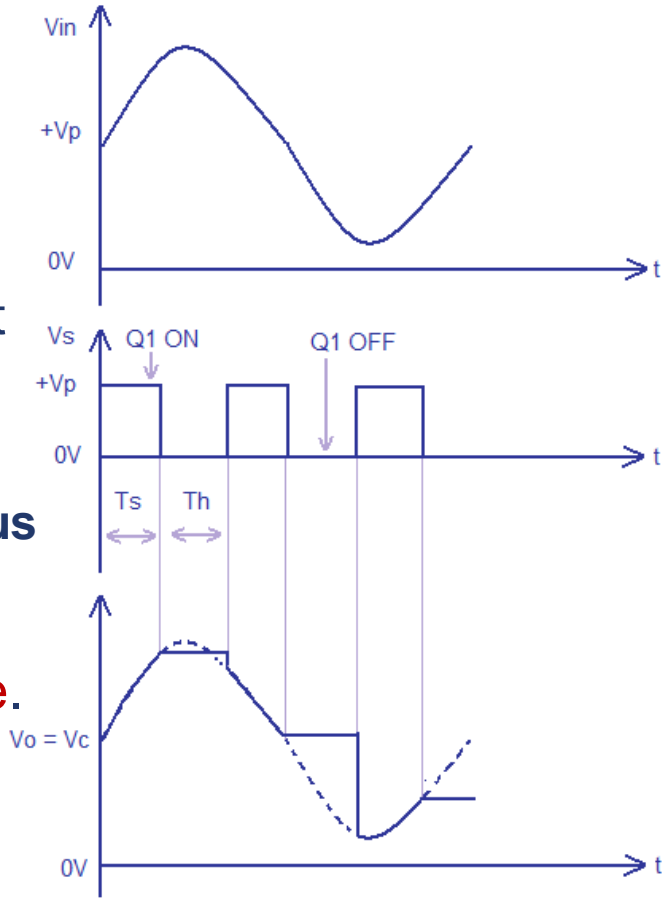


input and output waveforms

Flat-top Sampling

➢ Sample and Hold

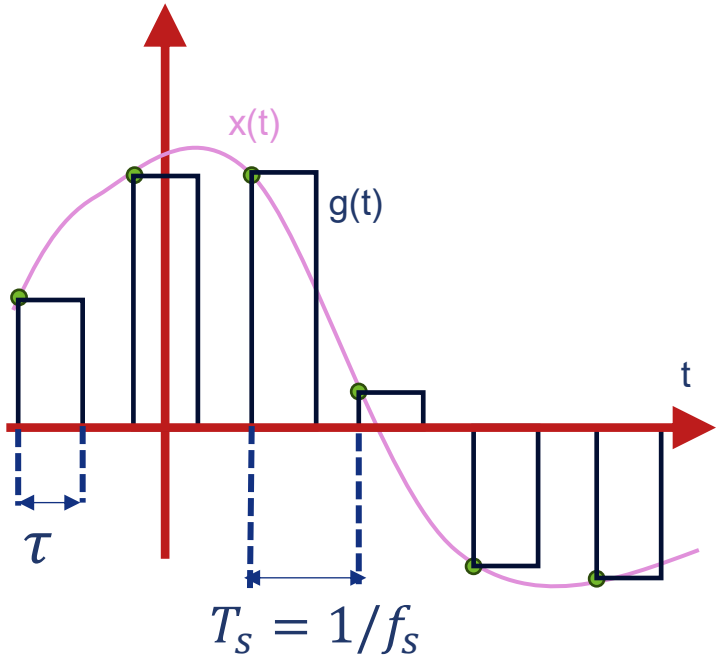
- ❖ The time during which sample and hold circuit generates the sample of the input signal is called sampling time.
  - ✓ Sampling time is generally between **1μs to 14 μs**
- ❖ The time duration of the circuit during which it holds the sampled value is called holding time.
  - ✓ The holding time can assume any value as required in the application



input and output waveforms

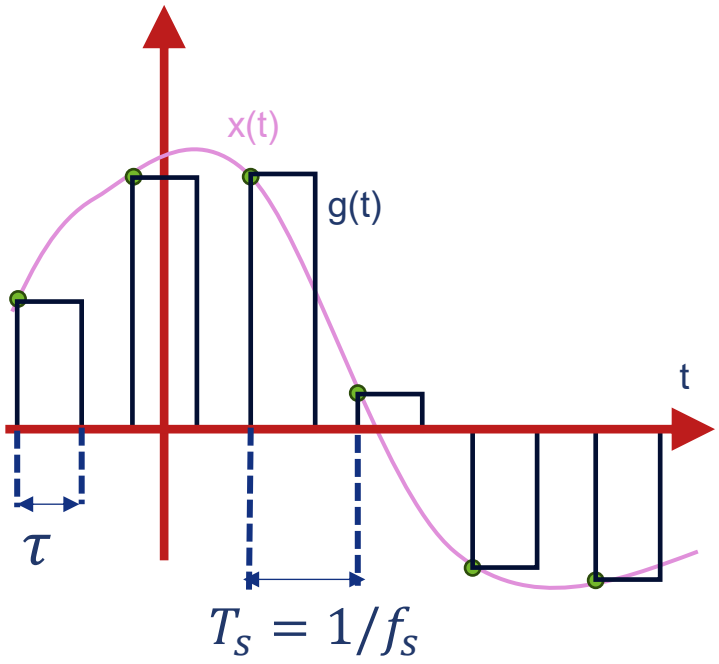
Flat-top Sampling

- Aperture effect
  - The amplitude of the flat top signal is *discretized*, but the original analog signal has *continuously varying* amplitude.
  - This leads to information loss during sampling
  - More particularly, high frequency components of the analog signal is lost



Flat-top Sampling

- Aperture effect
  - Aperture error is the difference between the actual value of the input signal, and the flat-topped sample value.
  - Aperture effect can be improved by reducing the holding time, or by using equalizer circuit to compensate for the distortion.



Different Types of Sampling Methods

1. Impulse Sampling

- The signal $x(t)$ is sampled by multiplying it with a train of Dirac impulses ($\delta(t - nT_s)$).
- Produces a series of instantaneous samples with zero duration.
- **Mathematically ideal** but **impossible to implement practically**

2. Natural Sampling

- The signal $x(t)$ is multiplied with a train of rectangular pulses of finite duration.
- Produces samples that closely follow the signal waveform during each sampling period.
- **Practical** but **sensitive to noise**

3. Flat-top Sampling

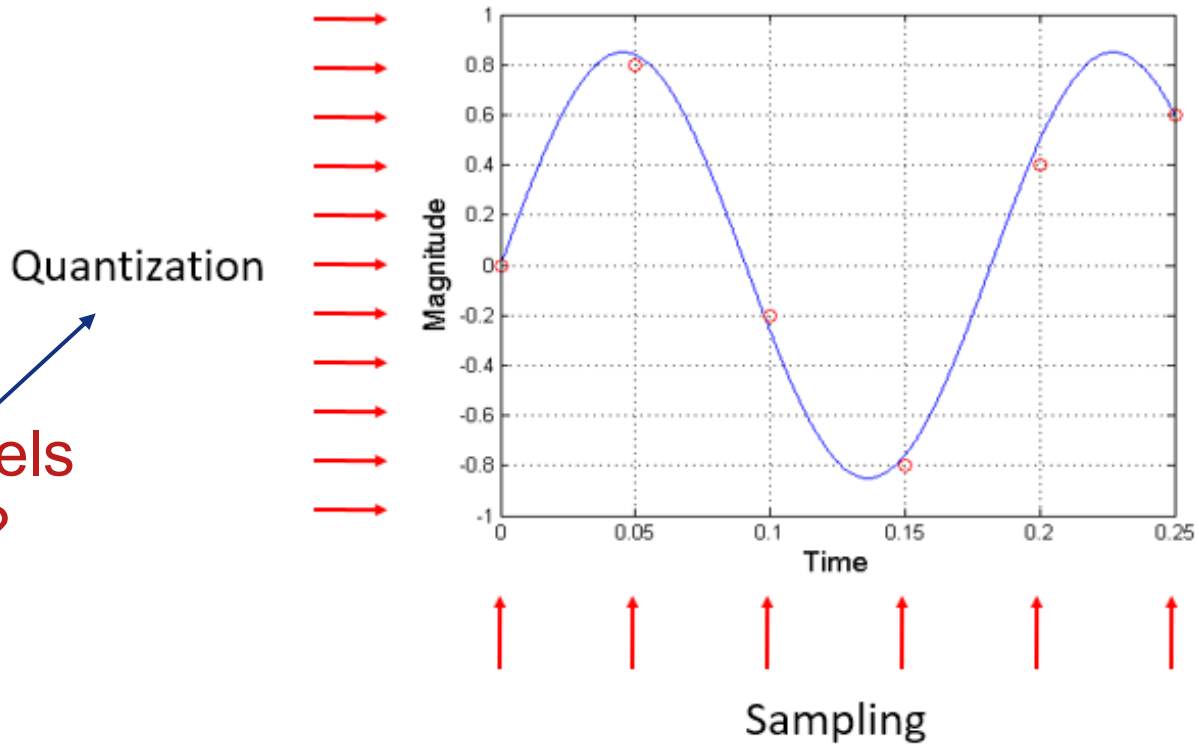
- The signal $x(t)$ is sampled and held constant over a duration using a sample-and-hold circuit.
- Produces a **staircase-like waveform** where each step equals the instantaneous value of the signal at the sampling instant.
- **Easy to implement** but causes **aperture effect**, distorting high-frequency components.

More on Sampling and Quantization

- Sampling
  - Impulse Sampling
  - Natural Sampling
  - Flat-top Sampling
- Quantization
  - Bit-based quantization
  - Quantization error

Quantization Levels

How many quantization levels should we take?

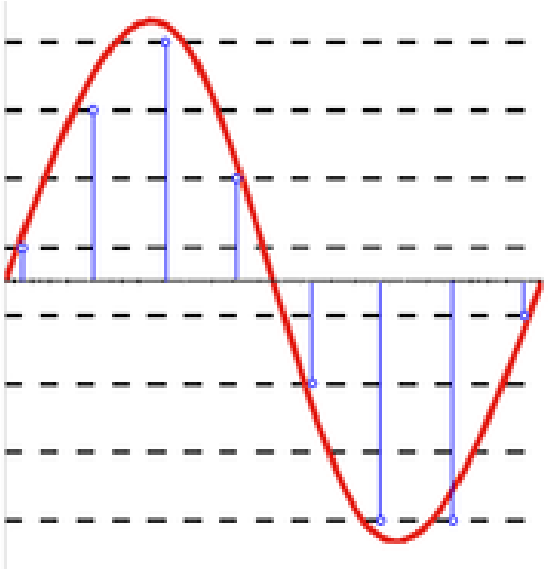


Quantization Basics – N-bit Quantization

- The number of quantization levels is often denoted as Q, measured in bits
  - 2-bits quantization→ $Q = 2^2 = 4$ levels
  - 3-bits→ $Q = 2^3 = 8$ levels
  - ..
  - 16-bits→ $Q = 2^{16} = 65{,}536$ levels
- In real systems, samples represent signal amplitudes (e.g., voltages).
- Each quantization level corresponds to a range of amplitudes mapped to a specific voltage value.
- 10-bit ADCs are widely used in systems like **microcontrollers** (e.g., Arduino) because they balance resolution and processing complexity.
- While not as precise as 10-bit ADCs, 8-bit ADCs are also used in applications like **video game controllers**, **basic sensors**, and **low-cost devices**.

Quantization Basics

2-bit quantization
(4 levels)



3-bit quantization
(8 levels)

Quantization Basics

- A practical system is often defined with a maximum output voltage V_{max} and a minimum output voltage V_{min}
- The lowest and highest quantization levels are **not** equal to V_{min} and V_{max} , but are instead the **midpoints** of the first and last quantization ranges.
- The distance between two quantization levels is the step size Δ

$$\Delta = \frac{V_{max} - V_{min}}{Q} = \frac{V_{max} - V_{min}}{2^N}$$

- The voltage a_n of each quantization level is

$$a_n = V_{min} + \left(n + \frac{1}{2}\right)\Delta\,, \qquad n = 0, \dots, Q-1$$

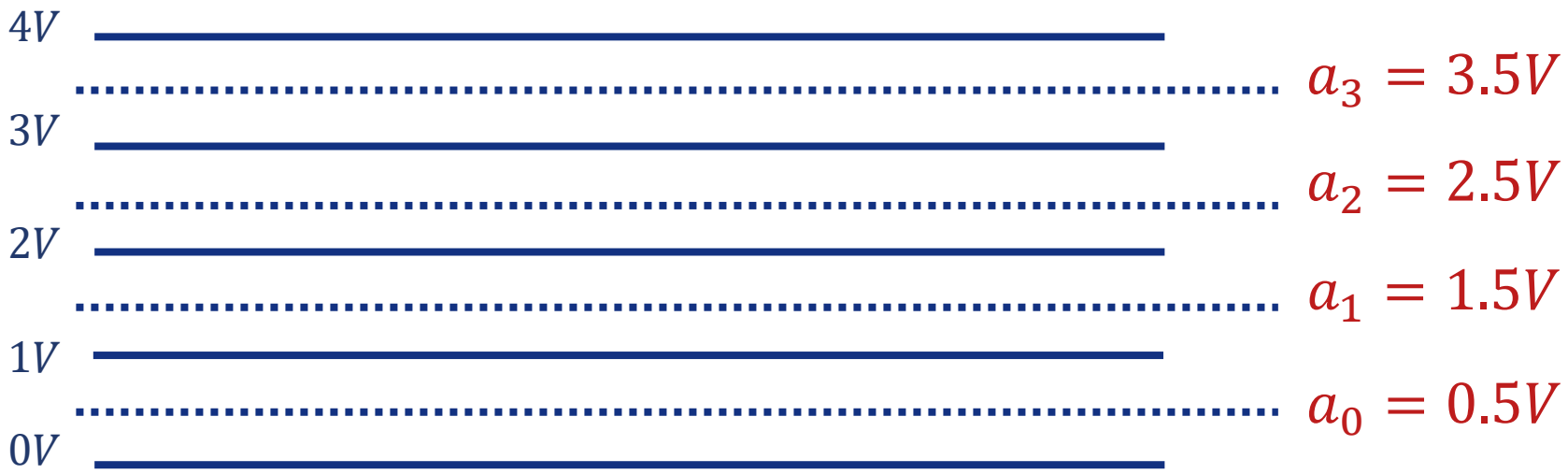
- This ensures uniform spacing and symmetric quantization error.

Quantization Basics - Example

- Example 1: A 2-bit quantization system is defined with $V_{max} = 4V$ and $V_{min} = 0V$. Compute the quantization levels.

Quantization Basics - Example

- <u>Example 1</u>: A 2-bit quantization system is defined with $V_{max} = 4V$ and $V_{min} = 0V$. Compute the quantization levels.



Quantization Basics - Example

- Example 2: A 2-bit quantization system is defined with $V_{max} = 4V$ and $V_{min} = 0V$. Use this system to quantize $[0.1, 0.4, 1.7, 3.1]$.

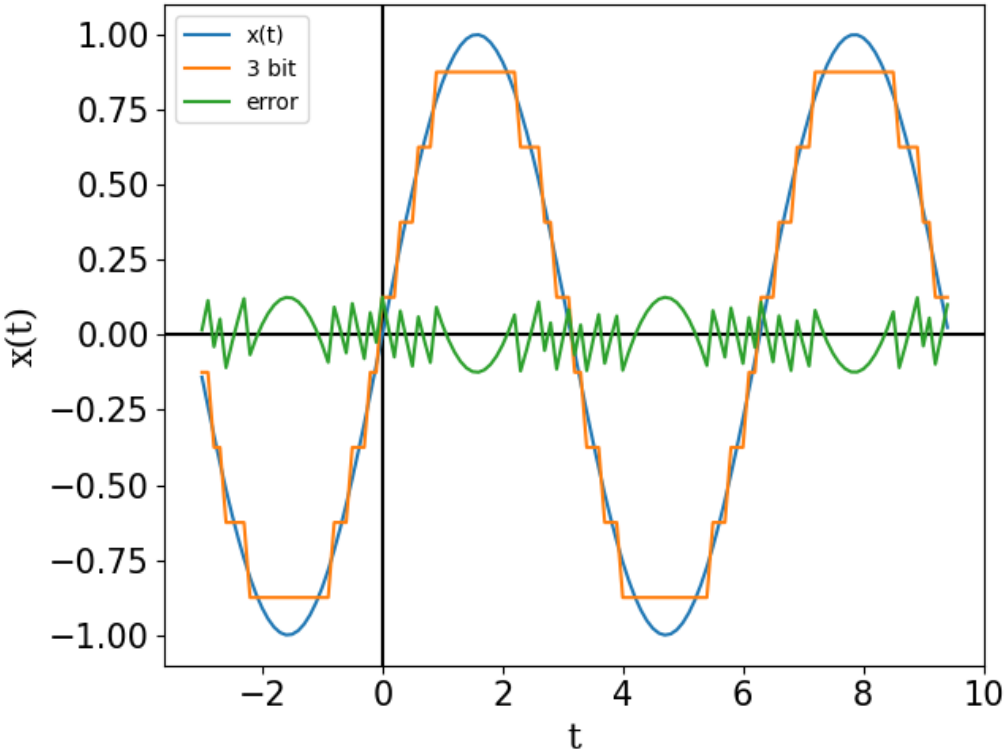
Quantization error

- Quantization error is the difference between the original signal and its quantized value:

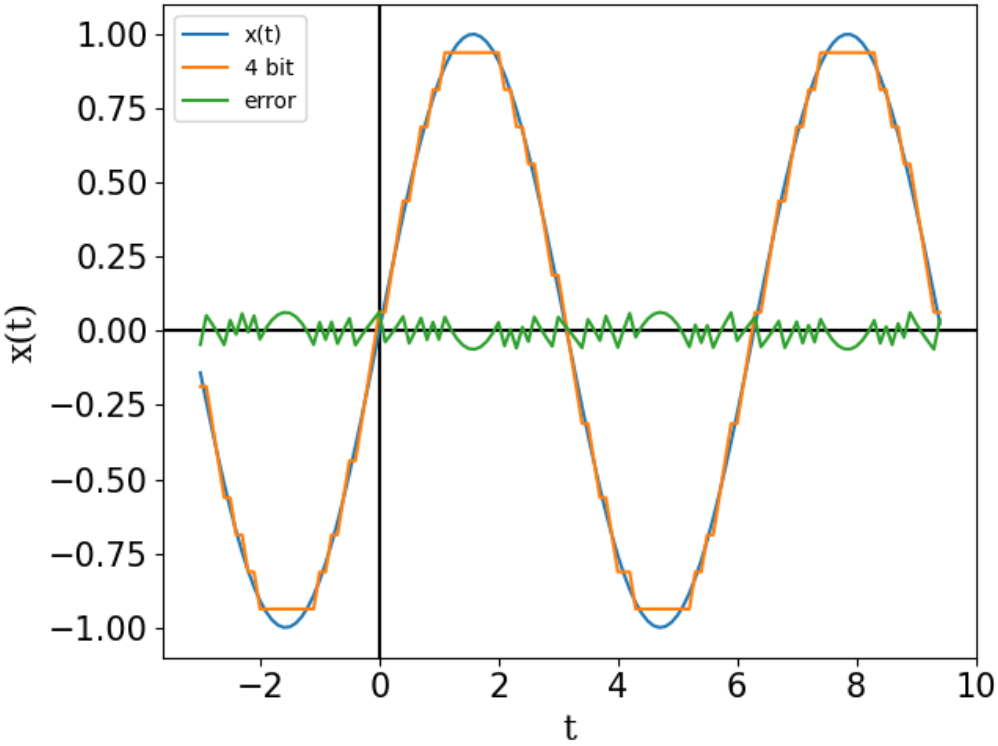
$$\epsilon = Q(x) - x$$

- The error is bounded by $-\frac{\Delta}{2} \leq \epsilon \leq \frac{\Delta}{2}$ on all quantization levels
- It can be **positive, negative, or zero**
- Placing levels at midpoints minimizes the maximum quantization error and ensures it is consistent across all intervals.

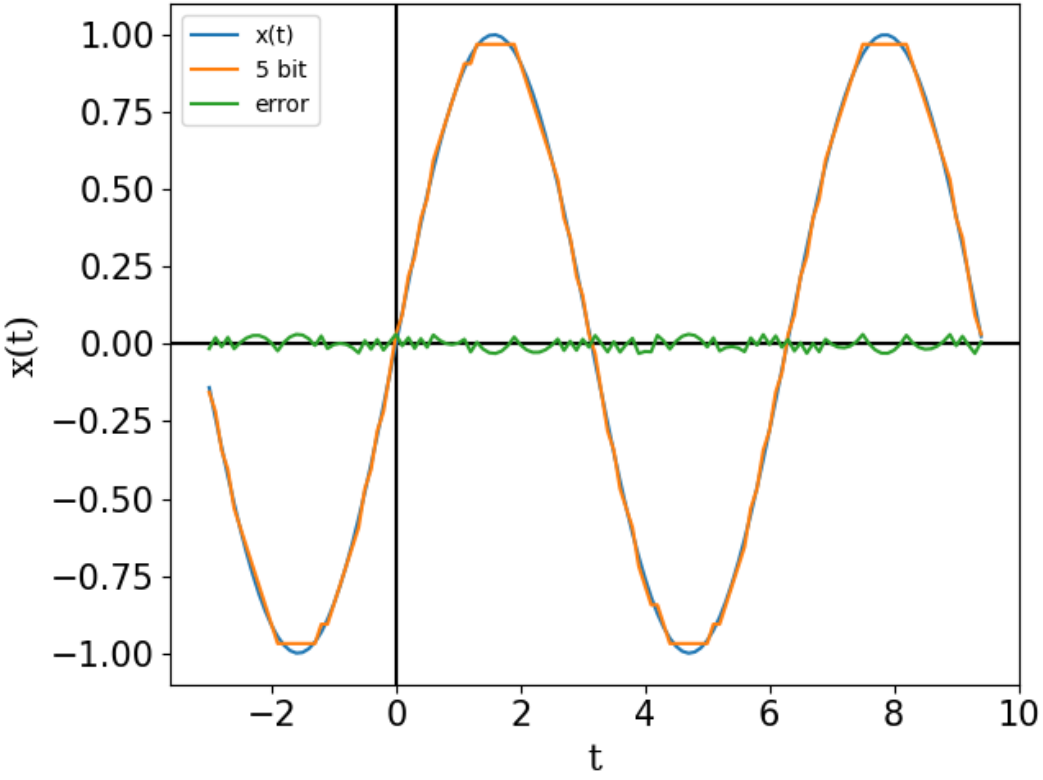
Quantisation error – sin wave 3 bit (8 level)



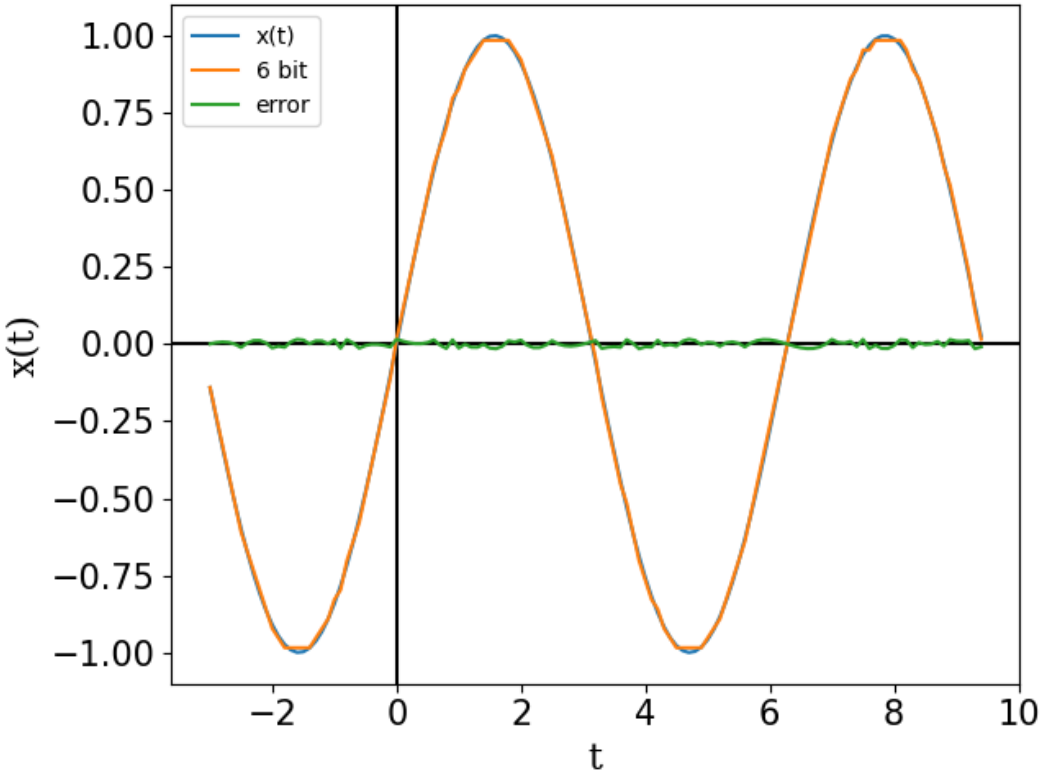
Quantisation error – sin wave 4 bit (16 level)



Quantisation error – sin wave 5 bit (32 level)



Quantisation error – sin wave 6 bit (64 level)



But note while we can hardly "see" this error it would still sound very bad on audio.

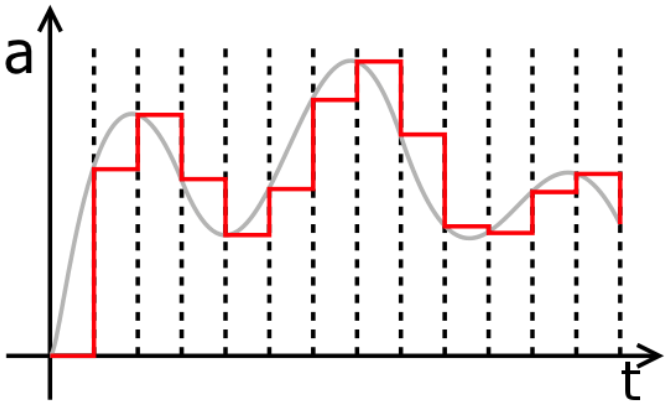
Quantization Example

- A temperature sensor converts a temperature range of $\mathbf{0°C}$ to $\mathbf{100°C}$ into an analog signal of **0V to 5V**, which is digitized using an **8-bit** quantization system.

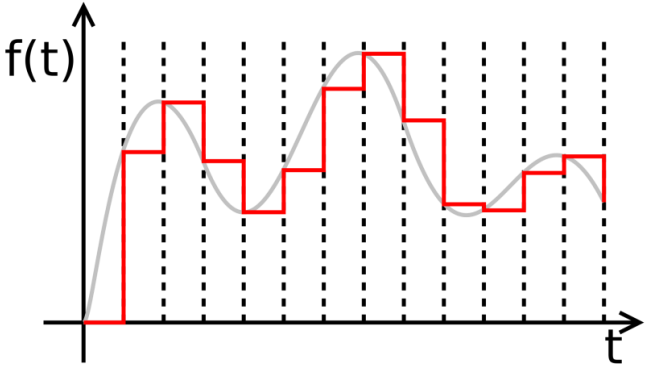
1. What is the **step size** (in volts) of this quantization system?
2. If the actual temperature is $24.2°C$, what is the **quantized temperature**?
3. What is the resulting **quantization error** in degrees Celsius?

Outline

- Analog to Digital Converter (ADC)
  - Sampling
  - Quantization
- Digital to Analog Converter (DAC)
  - Zero-order Hold



Zero-order Hold (ZOH)



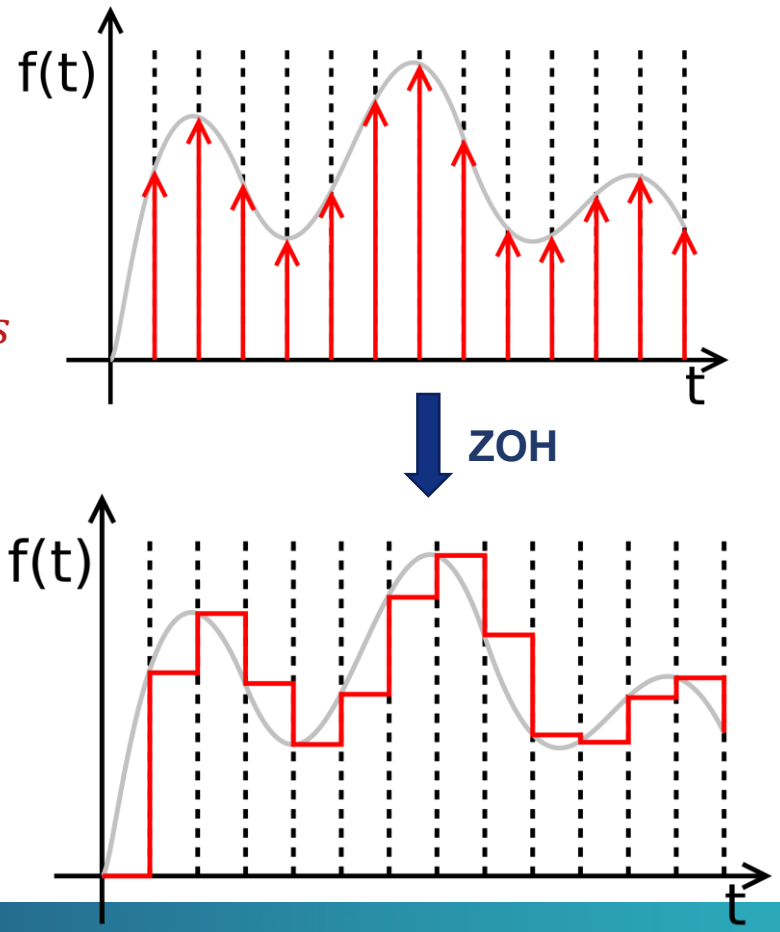
- **Zero-order hold**
  - It is a technique to convert digital signals to analog signals, for signal reconstruction
  - The continuous signal is reconstructed from its samples by holding the given sample for an interval until the next sample is received
  - All modern DACs use the zero-order hold technique

Zero-order Hold (ZOH)

- In general

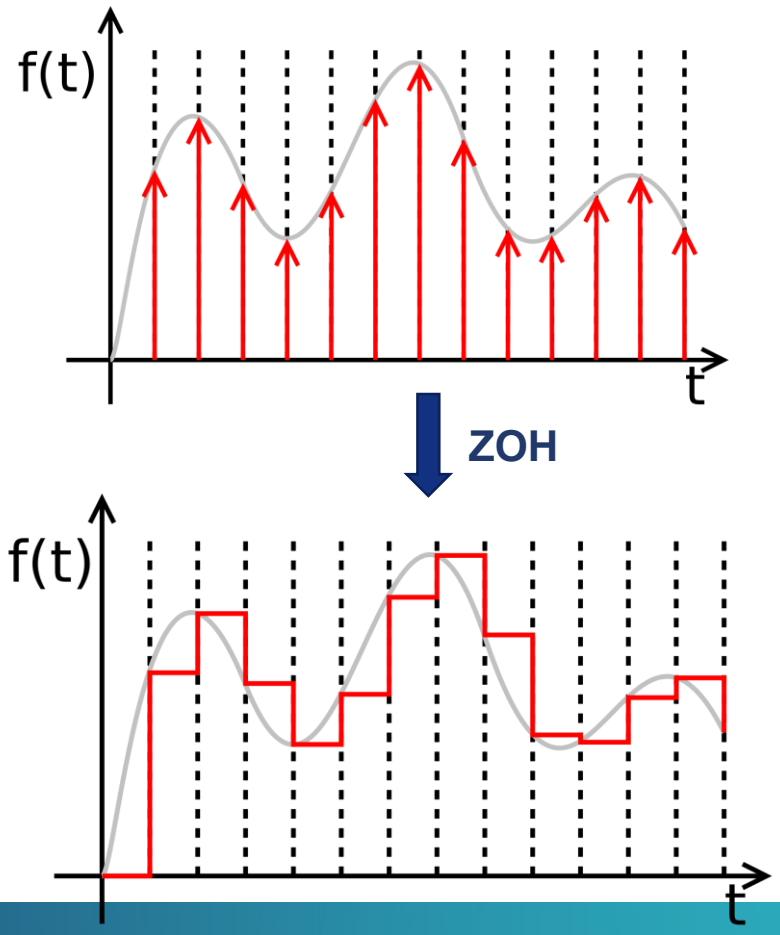
$$x(t) = x[n] \quad \text{for} \quad nT_s < t \leq (n+1)T_s$$

- For instance
  - $x(t) = x[0]$ for $0 < t \leq T_s$
  - $x(t) = x[1]$ for $T_s < t \leq 2T_s$
  - $x(t) = x[2]$ for $2T_s < t \leq 3T_s$
- T_s is the sampling interval



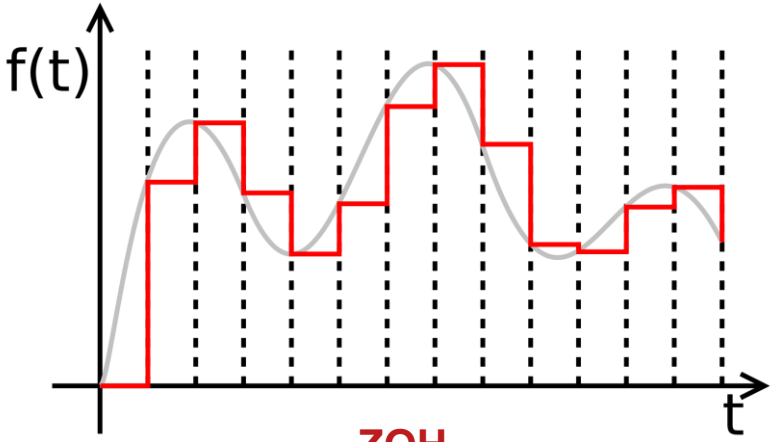
Zero-order Hold (ZOH)

- The accuracy of the ZOH reconstruction depends on the sampling rate
- The more we sample, the closer we approximate
- But sampling more costs more memory + processing power.

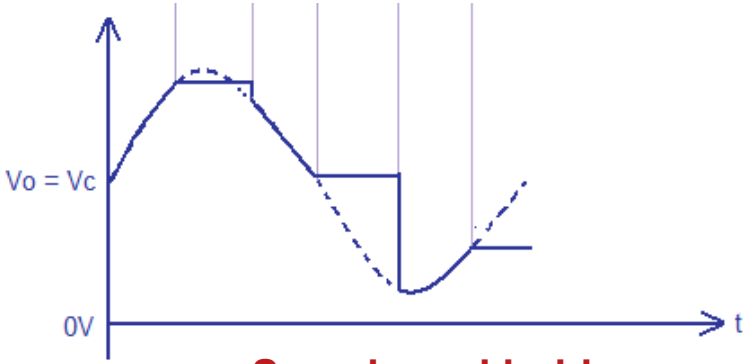


Zero-order Hold (ZOH)

- Relation to <u>sample and hold</u>
  - ZOH is used in DAC, sample and hold is used in ADC
  - ZOH inputs digital signals, sample inputs analog signals



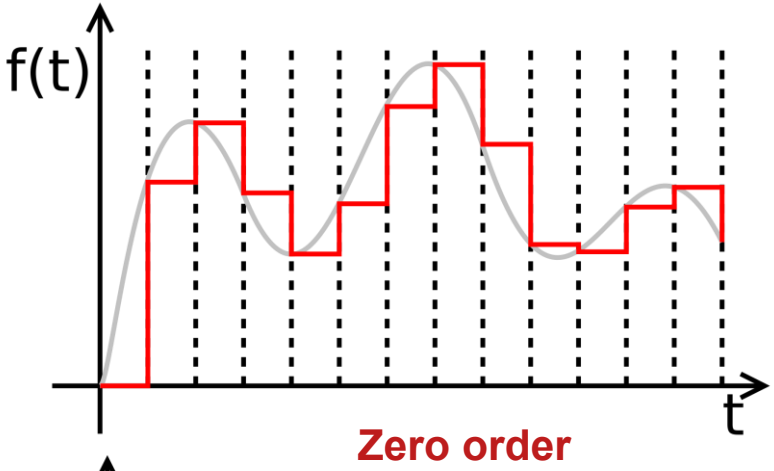
ZOH



Sample and hold

Zero-order Hold (ZOH)

- Why is it called ‘zero-order’?
  - Order: the degree of the Polynomial we can write the function with
  - **Zero Order** means the function is constant, we interpolate the same value in the missing parts
  - **First order** means we can use linear function to interpolate (Line with a slope).

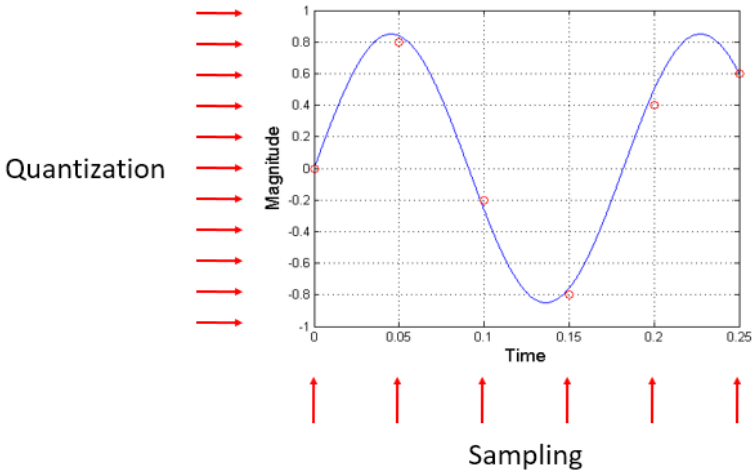


Zero order

First order

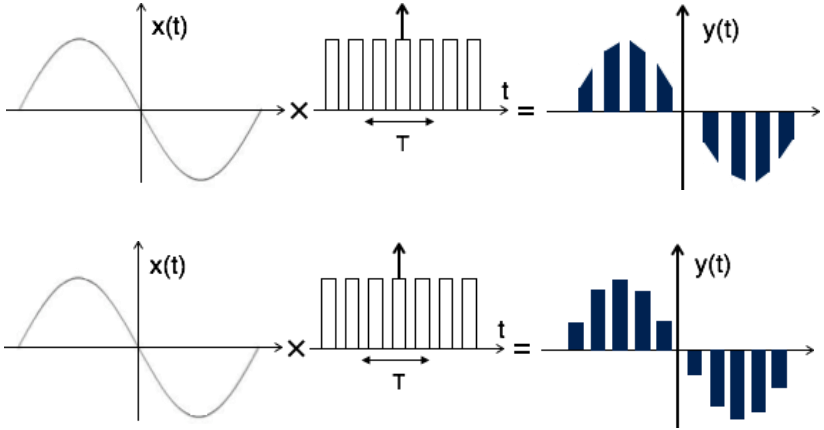
Summary

- We want to use computers to process real-world signal, but real-world signals are continuous in both time and amplitude
- Hence, we <u>discretize the time</u> axis via sampling, then <u>discretize the amplitude</u> axis via quantization
- Sampling + Quantisation = Digitisation



Summary

- Evaluating a signal at a time instant is known as the impulse/ideal sampling, which is impractical
- There are two types of practical sampling techniques:
  1. Natural sampling
  2. Flat-top sampling
     - ✓ more commonly used



Summary

- Quantization levels are represented by voltages
- The number of levels Q is often described in bits, i.e., N-bit quantization
- A quantization system often has a minimum and a maximum voltage
- But, the lowest quantization level is $V_{min} + \frac{1}{2}\Delta$
- The highest quantization level is $V_{max} - \frac{1}{2}\Delta$
- Here, $\Delta = \frac{V_{max} - V_{min}}{Q}$ is the step size between two quantization levels

Queen Mary
University of London

MATLAB – Sampling

✓ To sample a signal $x(t)$ in MATLAB, we type the equation of the signal, then evaluate it using different values of t based on the sampling interval T_s

<u>Example</u>

❑ Sample $x(t) = \sin(2\pi t)$ under the sampling interval $T_s = 0.1$ second

➢ <u>Method 1</u>: Calculate the values of $x(0.1), x(0.2), x(0.3), x(0.4), \ldots$

```
>> sin(2*pi*0.1)
ans =
    0.5878

>> sin(2*pi*0.2)
ans =
    0.9511

>> sin(2*pi*0.3)
ans =
    0.9511

>> sin(2*pi*0.4)
ans =
    0.5878
```

...

➢ Repeat as many times as the number of samples needed

MATLAB – Sampling

✓ To sample a signal $x(t)$ in MATLAB, we type the equation of the signal, then evaluate it using different values of t based on the sampling interval T_s

<u>Example</u>

❑ Sample $x(t) = \sin(2\pi t)$ under the sampling interval $T_s = 0.1$ second

➢ <u>Method 2</u>: Use vectorized calculation

```
>> sin(2*pi*[0.1 0.2 0.3 0.4 0.5 0.6])

ans =

    0.5878    0.9511    0.9511    0.5878    0.0000   -0.5878
```

➢ Simple one-line code~

MATLAB – Sampling

✓ To sample a signal $x(t)$ in MATLAB, we type the equation of the signal, then evaluate it using different values of t based on the sampling interval T_s

<u>Example</u>

❑ Sample $x(t) = \sin(2\pi t)$ under the sampling interval $T_s = 0.1$ second

➢ <u>Method 3</u>: Quicker vector creation using colon, ':'

```
>> 1:9

ans =

     1     2     3     4     5     6     7     8     9

>> 0.1:0.1:0.9

ans =

    0.1000    0.2000    0.3000    0.4000    0.5000    0.6000    0.7000    0.8000    0.9000
```

[start] : [step size] : [end]

MATLAB – Sampling

✓ To sample a signal $x(t)$ in MATLAB, we type the equation of the signal, then evaluate it using different values of t based on the sampling interval T_s

<u>Example</u>

❑ Sample $x(t) = \sin(2\pi t)$ under the sampling interval $T_s = 0.1$ second

➢ <u>Method 3</u>: Quicker vector creation using colon, ':'

```
>> t=0.1:0.1:0.9

t =

    0.1000    0.2000    0.3000    0.4000    0.5000    0.6000    0.7000    0.8000    0.9000

>> sin(2*pi*t)

ans =

    0.5878    0.9511    0.9511    0.5878    0.0000   -0.5878   -0.9511   -0.9511   -0.5878
```

Less codes, more output

Practice on your own

MATLAB – Sampling

✓ To sample a signal $x(t)$ in MATLAB, we type the equation of the signal, then evaluate it using different values of t based on the sampling interval T_s

<u>Exercises</u>

1. Obtain 5 samples of $x(t) = \sin(2\pi t)$ under the sampling interval $T_s = 0.15$ second
2. Obtain 50 samples of $x(t) = \sin(2\pi t)$ under the sampling interval $T_s = 0.15$ second
3. Obtain 10 samples of $x(t) = 10\sin(40\pi t)$ under the sampling rate $f_s = 40$ Hz
4. Obtain 10 samples of $x(t) = 10\cos(40\pi t)$ under the sampling rate $f_s = 10$ Hz
5. Sample $x(t) = \sin(2\pi t)$ every 0.1 second for 10 seconds
6. Sample $x(t) = \sin(0.5\pi t)$ every 0.05 second for 20 seconds